Informe Técnico — Segunda Entrega

Trabajo Práctico Integrador «Food Store» · Capa de datos (Unidades 4 a 8)

Base de Datos II · Tecnicatura Universitaria en Programación · UTN Facultad Regional Mendoza
Docente: Sergio Neira · Alumno: Máximo Juan Cruz Quiroga (maximoj.quiroga@alumnos.frm.utn.edu.ar)
Noviembre de 2026

Las cifras, tiempos y resultados citados provienen exclusivamente de los scripts, registros y evidencias versionados en el repositorio `foodstore_github`; la carpeta de origen se indica en cada caso.

1. Implementaciones por Unidad (4 a 8)

La capa de datos de Food Store se construyó como una arquitectura de persistencia políglota: PostgreSQL 17 (versión que satisface el requisito 16+) actúa como sistema de registro transaccional; MongoDB aloja el historial documental de pedidos, y SQLite funciona como base local desconectada de la tablet de venta en feria.

Unidad	Motor	Elementos integrados
U4 · FNBC y desnormalización controlada TP6_FNBC_Desnormalizacion	PostgreSQL	Descomposición de `control_lote_almacen` en `responsable_deposito` y `control_lote`, con vista de reconstrucción `v_control_lote_almacen`. Borrado lógico (`eliminado`) en `pedidos` y `detalles_pedido`. Columnas caché `fecha_pedido_cache`, `id_categoria_cache` y `pedido_eliminado_cache` (`NOT NULL`, FK a `categorias`), índice `idx_TP6_detalle_fecha_cache` y tres *triggers* de sincronización.
U5 · Administración y seguridad TP7_Administracion_y_Seguridad	PostgreSQL	Cinco roles de grupo (`NOLOGIN`) y dos roles de conexión (`app_web`, `admin_datos`); revocación total a `PUBLIC` y `ALTER DEFAULT PRIVILEGES`. Funciones `fn_autenticar` y `fn_resetear_password` (`SECURITY DEFINER`, `search_path` fijo, hash bcrypt vía `pgcrypto`). Vistas de exposición controlada (`v_usuario_publico`, `v_reporte_ventas_cliente`) y vista materializada `mv_resumen_ventas_categoria_mes`. Auditoría nativa por parámetros de registro (`log_connections`, `log_disconnections`, `log_statement = all`). Anonimización en `usuarios_anon`.
U6 · Mantenimiento, respaldo y recuperación TP8_Mantenimiento_Respaldo_Recuperacion	PostgreSQL	Respaldo lógico `pg_dump -Fc` con verificación `pg_restore --list`, tamaño y SHA-256 (`backup_foodstore.ps1`); restauración automatizada y cronometrada sobre base descartable (`restore_test.ps1`); diagnóstico de salud de solo lectura en 9 secciones (`health_check.sql`); mantenimiento justificado con `ANALYZE` focalizado (`maintenance.sql`); suite simulada de 14 verificaciones (`pruebas/test_delivery.ps1`).
U7 · NoSQL nosql/TP9_MongoDB_NoSQL	MongoDB	Colección `foodstore_nosql.pedidos` con patrón embebido (subdocumento `cliente` y arreglo `lineas`) conservando los identificadores de PostgreSQL. Validación `$jsonSchema` (tipos, enumeraciones, `cantidad ≥ 1`, `precio_unitario ≥ 0`) y validación cruzada `$expr` (total = Σ subtotales). Índices: `id_pedido` único, `estado`, `lineas.id_producto` (multiclave) y `total`. Migración por `migracion_pedidos.js` y ETL en Python (`etl_postgres_a_mongo.py`).
U8 · Móvil y cierre de la capa de datos nosql/TP10_Movil_SQLite	SQLite + PostgreSQL	Esquema de seis tablas `STRICT` con `PRAGMA foreign_keys = ON`: `producto_cache`, `snapshot_servidor`, `snapshot_servidor_import`, `pedido_borrador`, `item_borrador` y `cola_sync`. Precios en centavos enteros. Importación de la instantánea exportada por `v_snapshot_movil` (PostgreSQL), recarga por `UPSERT`, semáforo de sincronización (`semaforo.sql`) y casos de prueba C2–C6 (`casos_2c.sql`).

2. Metodología de Pruebas

2.1 Transacciones reversibles

Las validaciones con efectos de escritura se ejecutaron dentro de bloques `BEGIN … ROLLBACK`, de modo que cada script resulta idempotente y no altera la base de laboratorio: descomposición FNBC, desnormalización (incluido un `SAVEPOINT` con modificación forzada de la fecha de un pedido para auditar la propagación por *trigger*), simulacro de incidente y anonimización. El diagnóstico de salud de U6 se ejecuta en `BEGIN TRANSACTION READ ONLY … ROLLBACK`. En SQLite, `pruebas_sqlite.sql` contrasta una transacción confirmada contra una revertida y exige conteos nulos tras el `ROLLBACK`.

2.2 Especificaciones formales (Kiro)

En U5, la especificación `docs/kiro_roles_spec` (requisitos RF-01 a RF-07, diseño y `verificacion_permisos.md`) fijó la matriz de privilegios esperada, contra la cual se verificó el catálogo del motor. En U8, `docs/kiro/` contiene cuatro requisitos en notación EARS (REQ-01 a REQ-04) mapeados uno a uno con los casos C2, C3, C4 y C6; esta especificación se redactó de forma retroactiva y se verificó línea por línea contra `semaforo.sql`. Kiro no se utilizó en U6.

2.3 Copias de prueba

- **U4:** pruebas sobre la base de trabajo `foodstore_copia_trabajo`.
- **U5:** respaldos previos en formato *custom* antes de cada etapa (`…_antes_tp7_instalacion.dump`, 17.909 bytes; `…_antes_tp7_parte_d.dump`, 76.543 bytes).
- **U6:** las bases de origen (`foodstore_dev`, `foodstore_tp3_carga`) se trataron como solo lectura; toda restauración y mantenimiento se aplicó en `foodstore_restore_test`, creada desde `template0`.
- **U8:** base SQLite `foodstore_movil.db`, regenerable con `simulador.py`, alimentada desde una instantánea CSV del servidor.

2.4 Simulacro de acceso indebido (U5)

El script `simulacro_incidente.sql` reprodujo, con registro nativo activo, la secuencia de un atacante: tres autenticaciones fallidas contra `fn_autenticar` para la cuenta `admin@foodstore.utn.edu.ar`; un inicio de sesión exitoso; una lectura masiva de usuarios activos, y un intento de escalamiento de privilegios desde la sesión de aplicación (`SET LOCAL ROLE app_web; GRANT admin_datos TO app_web;`). El log resultante se anonimizó y se entregó como única evidencia a una sesión limpia de OpenCode (Gemini) para contrastar su reconstrucción con la cronología real.

2.5 Simulacro de desastre (U6)

1. Captura de la línea base en el origen: conteos de las tablas canónicas y nueve controles lógicos de integridad (precios y stock no negativos, cantidades positivas, subtotales correctos y ausencia de huérfanos).
2. Respaldo con `pg_dump -Fc`, autenticación desatendida (`-w` + `pgpass.conf`) y almacenamiento fuera del repositorio.
3. Verificación del archivo: legibilidad (`pg_restore --list`), tamaño mayor que cero y hash SHA-256 registrado en `metadata.json`.
4. Guardas: aborta si el destino ya existe (sin borrado automático) o si el SHA-256 no coincide.
5. Inicio del cronómetro, `createdb --template=template0` y `pg_restore --exit-on-error --no-owner --no-privileges`.
6. Comparación tabla por tabla contra la línea base, re-ejecución de los nueve controles, cierre del cronómetro (RTO) y emisión de `result.json`.
7. Diagnóstico de salud y mantenimiento sobre la copia restaurada.

2.6 Simulación de conflictos de sincronización (U8)

Se aplicó la regla «el móvil propone, el servidor dispone»: la tablet solo arma borradores y encola solicitudes en `cola_sync`. El semáforo evalúa únicamente las solicitudes `PENDIENTE` contra la instantánea del servidor, con precedencia estricta: **rojo** (`RECHAZADO`: producto inexistente, no disponible, dado de baja o con stock insuficiente) por sobre **amarillo** (`CONFLICTO`: precio observado distinto) y este por sobre **verde** (`SIMULADO_APROBADO`, que es una prevalidación y no una venta). La idempotencia se garantiza con `solicitud_uuid` como clave primaria. Los casos C2 a C6 se prepararon en el servidor con `preparar_casos_2c.sql` y en la tablet con `casos_2c.sql`.

3. Resultados Obtenidos e Informe Post-Incidente

3.1 U4 — Desnormalización del «Top 5 diario de categorías»

Consulta	Ronda 1	Ronda 2	Promedio	Búferes leídos
Normalizada (JOIN pedidos–detalles–productos)	49,291 ms	47,556 ms	48,42 ms	10.942
Desnormalizada (columnas caché + índice)	1,980 ms	1,323 ms	1,65 ms	565

La mejora es de aproximadamente **29 veces** (−96,6 % de tiempo) con un 94,8 % menos de lecturas lógicas. La equivalencia de resultados entre ambas consultas se verificó con `EXCEPT` (0 filas de diferencia), y la sincronización inicial actualizó 499.571 filas de `detalles_pedido`. En FNBC, la reconstrucción de la relación original mediante `EXCEPT` bidireccional arrojó 0 diferencias.

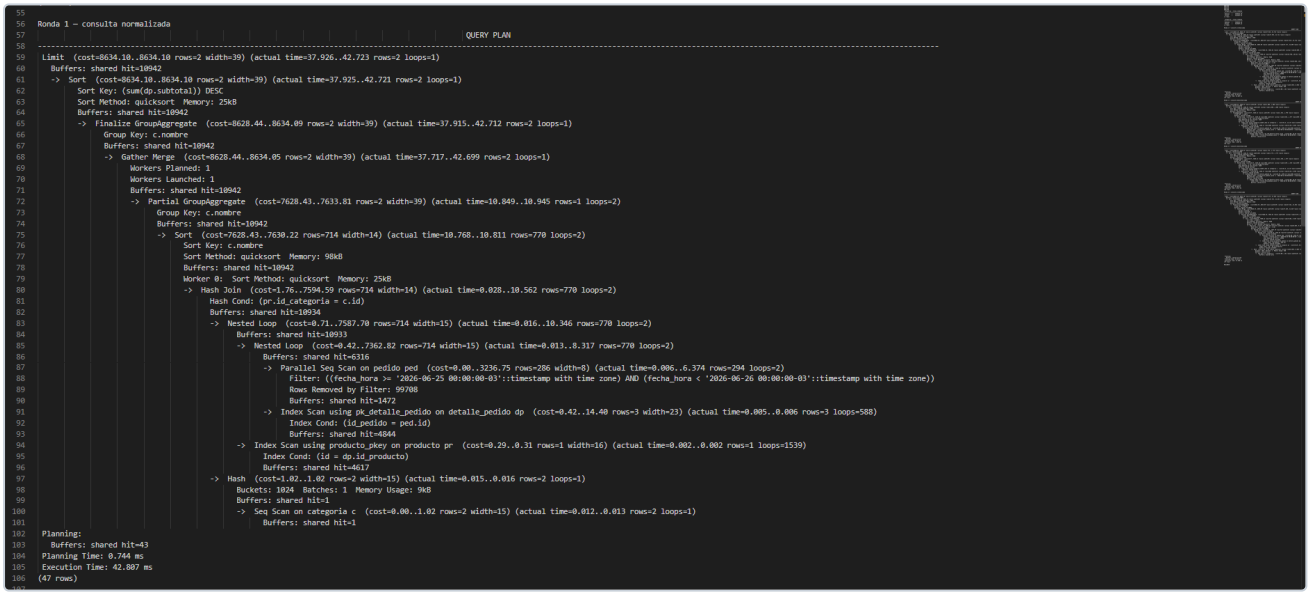


Fig. 1a. `EXPLAIN (ANALYZE, BUFFERS)` de la consulta normalizada: *Parallel Seq Scan* sobre pedidos, bucles anidados y 10.942 búferes compartidos.



```
Ronda 1 — consulta desnormalizada
                                                
```

3.3 U6 — Simulacro de desastre

Tiempo exacto de restauración (RTO observado, validación final):

- `foodstore_dev` : **2,5788433 s** (≈ 2,58 s), con un volcado de 46.877 bytes.
- `foodstore_tp3_carga` : **9,1510594 s** (≈ 9,15 s), con un volcado de 7.321.779 bytes y 769.584 filas en las tablas principales.

Meta de la especificación (SPEC_U6): RTO ≤ 15 min y RPO ≤ 24 h. Pérdida efectiva de datos: **nula**.
Estado: **PASS**.

Un ensayo previo, registrado en `evidencias/results.json` y `summary.csv`, obtuvo 2,98 s y 8,19 s respectivamente. La antigüedad del respaldo al iniciar el simulacro (aproximación del RPO) fue de 100,36 s (dev) y 49,96 s (carga) en la validación final.

Tabla	dev: origen	dev: restaurada	carga: origen	carga: restaurada
categorias	2	2	2	2
clientes	3	3	20.003	20.003
productos	3	3	50.003	50.003
pedidos	5	5	200.005	200.005
detalles_pedido	7	7	499.571	499.571

Los nueve controles lógicos arrojaron 0 anomalías en todos los ensayos. El diagnóstico de salud y el mantenimiento finalizaron con código de salida 0; el único contenido de `maintenance.stderr.log` son mensajes `INFO` de `ANALYZE VERBOSE`, que `psql` emite por la salida de error. La suite simulada reportó **PASS 14 checks**.

```
Base               Destino               Estado RTO_segundos RPO_proxy_seg Dump_bytes
----               -------               ------ ------------ ------------- ----------
foodstore_dev      foodstore_restore_test PASS          2,98          0,78      46877
foodstore_tp3_carga foodstore_restore_test PASS         8,19        272,61    7321779
```

Fig. 4. Resumen de restauración del ensayo previo (`evidencias/summary.csv`): ambas bases con estado PASS, RTO de 2,98 s y 8,19 s.

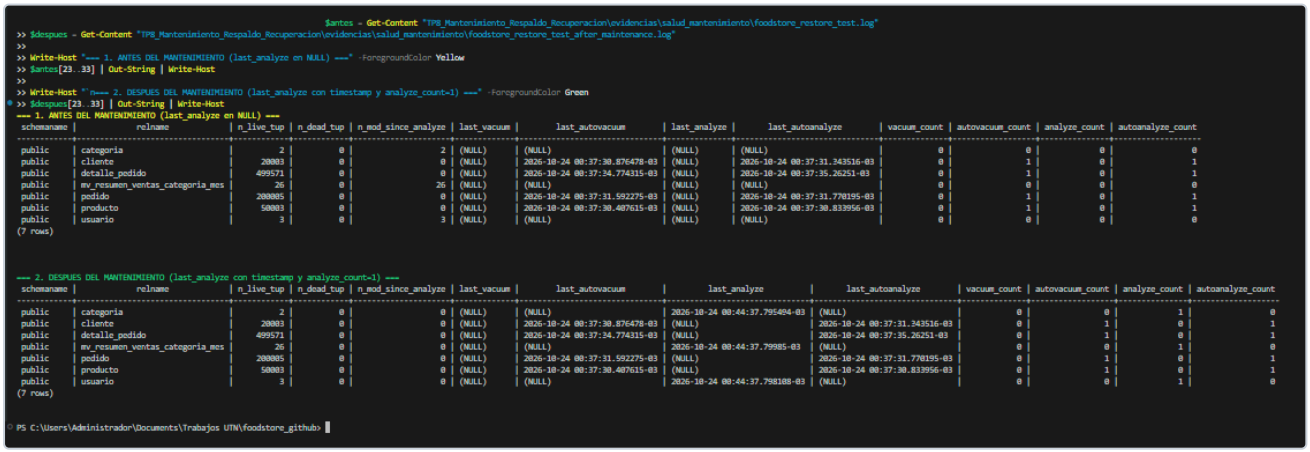
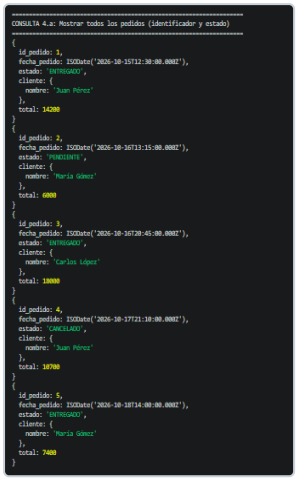


Fig. 5. `pg_stat_user_tables` en `foodstore_restore_test` antes y después de `maintenance.sql` : `categoria` , `usuario` y `mv_resumen_ventas_categoria_mes` pasan de `last_analyze` nulo a una marca temporal, con `analyze_count = 1` .

3.4 U7 — Equivalencia PostgreSQL / MongoDB

Sobre los 5 pedidos migrados, ambos motores devolvieron los mismos identificadores: estado `ENTREGADO` → `[1, 3, 5]` ; producto `id_producto = 1` → `[1, 2, 4]` ; total > $10.000 → pedidos 3, 1 y 4. El validador `$expr`

rechazó los documentos con total alterado o subtotal inconsistente (código 121, `Document failed validation`).



(a) 4.a · todos los pedidos



(b) 4.b · estado ENTREGADO



(c) 4.d · total > $10.000



(d) 4.c · `lineas.id_producto = 1`



(e) 4.c · variante con `$filter`

Fig. 6. Ejecución de `consultas_mongodb.js` sobre `foodstore_nosql.pedidos`. Los identificadores devueltos coinciden con los de la versión relacional.

3.5 U8 — Conflictos de sincronización

Caso	Situación	Estado resultante	Motivo registrado
C2-a	Precio visto $4.500; servidor $4.800	`CONFLICTO`	Cambió el precio observado
C2-b	El cliente acepta el precio nuevo (`c2-solicitud-2`)	`SIMULADO_APROBADO`	Precio, disponibilidad y stock coinciden
C3	Pedido de 99 unidades con stock 35 en el servidor	`RECHAZADO`	Stock insuficiente
C4	Producto con baja lógica en el servidor	`RECHAZADO`	Producto no disponible
C5	Catálogo y precios vigentes	`SIMULADO_APROBADO`	Precio, disponibilidad y stock coinciden
C6	Reinserción de un `solicitud_uuid` existente	Rechazo por PK	Sin duplicación ni reevaluación

La ejecución automatizada con `simulador.py` (SQLite 3.50.4) confirmó además las pruebas previas: `COMMIT` persistente, `ROLLBACK` sin filas huérfanas, consulta desconectada del catálogo (C1) y rechazo por `CHECK` de una cantidad igual a cero y por FK de un producto inexistente (C7).

```
TP10 (UNIDAD 8): SIMULADOR DEL SEMÁFORO Y BASE DE LA TABLET (SQLite)
Alumno: Máximo Juan Cruz Quiroga | Base de Datos II | UTN FRM
Motor SQLite: v3.50.4
============================================================

[1/5] Creando esquema de tablas STRICT en SQLite...
  [OK] Tablas producto_cache, snapshot_servidor, pedido_borrador, item_borrador, cola_sync creadas.

[2/5] Importando instantánea inicial del servidor (servidor_snapshot.csv)...
  [OK] Instantánea cargada en snapshot_servidor y replicada en producto_cache.

[3/5] Ejecutando pruebas unitarias de SQLite (1.c: COMMIT, ROLLBACK, C1, C7)...
  [OK] Prueba COMMIT: Borrador e ítems persistidos correctamente.
  [OK] Prueba ROLLBACK: Transacción descartada sin dejar filas huérfanas.
  [OK] Caso C1 (Consulta Offline): 4 productos disponibles consultados sin red.
  [OK] Caso C7 (Restricción CHECK): Rechazó correctamente ítem con cantidad = 0.
  [OK] Caso C7 (Clave Foránea): Rechazó correctamente producto inexistente (id=9999).

[4/5] Simulando Casos C2 a C6 del Semáforo con Instantánea Actualizada...
  [OK] Servidor actualizó catálogo: Hamb. Clásica subió a $4.800 y Papas Fritas fue dada de baja.
  [OK] Caso C6 (Idempotencia / PK): Rechazó correctamente UUID repetido ('c5-solicitud').

[5/5] Tabla de Resultados del Semáforo en la Tablet:
-----------------------------------------------------------------------------------------------
Caso   | Solicitud UUID     | Estado Semáforo    | Motivo / Explicación
-----------------------------------------------------------------------------------------------
C2-a   | c2-solicitud-1     | CONFLICTO          | Cambió el precio observado
C2-b   | c2-solicitud-2     | SIMULADO_APROBADO  | Precio, disponibilidad y stock coinciden
C3     | c3-solicitud       | RECHAZADO          | Stock insuficiente
C4     | c4-solicitud       | RECHAZADO          | Producto no disponible
C5     | c5-solicitud       | SIMULADO_APROBADO  | Precio, disponibilidad y stock coinciden
-----------------------------------------------------------------------------------------------
```

Fig. 7a. Salida de `simulador.py`: creación del esquema `STRICT`, importación de la instantánea, pruebas unitarias y tabla de resultados del semáforo (C2–C6).

```
      tabla      | total_registros
-----------------+-----------------
 pedidos         |          400000
 detalles_pedido |          609996
(2 rows)


 id_producto |     nombre     | stock | precio_actual | activo
-------------+----------------+-------+---------------+--------
           1 | Producto_Gen_1 |   580 |       4398.82 | t
           2 | Producto_Gen_2 |   567 |       4023.57 | t
           3 | Producto_Gen_3 |   338 |       1587.81 | t
           4 | Producto_Gen_4 |   403 |       2848.66 | t
(4 rows)
```

Fig. 7b. Salida de `postgresql/verificar_sin_cambios_2c.sql`: conteo de pedidos y detalles, y stock de productos del servidor consultados después de la simulación. La tablet no ejecuta escrituras sobre PostgreSQL.

3.6 Informe post-incidente

A. Incidente de seguridad simulado (U5). La cronología se verificó en el log nativo (PID 21348). El patrón de ataque detectado fue: fuerza bruta (3 fallos) → acceso exitoso → lectura masiva → intento de escalamiento. La contención fue efectiva a nivel de motor, porque el diseño de mínimo privilegio impidió el escalamiento. La IA recomendó bloquear la IP de origen; esa recomendación se descartó por el riesgo de denegar el servicio a usuarios legítimos detrás de NAT o proxy. Las acciones adoptadas fueron aislar la sesión, forzar el restablecimiento de la contraseña mediante `fn_resetear_password`, auditar las trazas previas y verificar las membresías de roles. Quedó como limitación que los respaldos de U5 se validaron solo por cabecera (`pg_restore --list`), sin restauración completa; esa brecha se cerró con el simulacro de U6.

B. Simulacro de desastre (U6).

- **Causas raíz identificadas:** (1) fallo de autenticación desatendida (`fe_sendauth: no password supplied`) por formato o permisos incorrectos de `pgpass.conf`; (2) falsos positivos al comparar metadatos JSON como texto, por diferencias en el orden de las claves; (3) tablas pequeñas sin estadísticas (`last_analyze` nulo), porque no alcanzan el umbral de autovacuum.
- **Impacto:** respaldos iniciales abortados (`FAIL`), una falsa alarma de verificación y riesgo de planes subóptimos. Impacto en las bases de origen: nulo, sin pérdida de datos.
- **Lecciones aprendidas:** `pgpass.conf` debe tener formato exacto y permisos NTFS restringidos; los metadatos deben compararse deserializados, propiedad por propiedad; ninguna eliminación de bases debe automatizarse.
- **Acciones correctivas propuestas:** ejecutar `ANALYZE` automático al final de `restore_test.ps1` e incorporar `pg_dumpall --globals-only` para respaldar roles y privilegios globales.

4. Decisiones de Diseño y Arquitectura

4.1 Refactorización hasta FNBC y desnormalización controlada

FNBC. La relación R(L, D, C) —lote, depósito, responsable de control— presenta las dependencias LD → C y C → D, con claves candidatas LD y LC. Cumple 3FN porque D es primo, pero **viola FNBC**, ya que el determinante C no es superclave. Se descompuso en R_1(C, D) = `responsable_deposito` y R_2(L, C) = `control_lote`. La reunión es **sin pérdida** ($R_1 \cap R_2$ = {C}, clave de R_1), pero la dependencia LD → C **no se preserva localmente**: abarca ambas tablas y las restricciones de clave no impiden asignar dos responsables del mismo depósito a un mismo lote. Se aceptó ese costo, conocido en la teoría de FNBC, a cambio de eliminar la redundancia C → D; en producción requiere una aserción implementada mediante *trigger*.

Desnormalización.

- **Cuándo:** solo para la consulta «Top 5 diario de categorías por facturación», que se resuelve en tiempo real sobre 200.000 pedidos.
- **Por qué:** el plan normalizado requería un *Parallel Seq Scan* y bucles anidados con 10.942 búferes leídos (48,42 ms). Se descartó una vista materializada porque el desfase entre refrescos no es admisible para un ranking en tiempo real.
- **Salvaguardas:** tres *triggers* transaccionales mantienen las copias ante cambios en `detalles_pedido`, `pedidos` y `productos`; las columnas caché son `NOT NULL` con FK a `categorias`; un índice B-Tree cubre `fecha_pedido_cache`, y una verificación `EXCEPT` confirma la equivalencia con la versión normalizada. La redundancia queda acotada a una tabla y a una consulta; el costo se traslada a las escrituras, que asumen la propagación.

4.2 Roles bajo el principio de mínimo privilegio

Rol	Privilegios	Justificación
`rol_app_lectura`	`SELECT` sobre 5 tablas de negocio y 3 vistas de catálogo	Consulta del catálogo y las ventas sin capacidad de modificación ni acceso a `usuarios`.
`rol_app_escritura`	`INSERT` y `UPDATE` (sin `DELETE`); uso de secuencias; `EXECUTE fn_autenticar`	La baja es lógica (regla R7): sin `DELETE` ni `TRUNCATE` se protege el historial contable.
`rol_soporte`	`SELECT` por columnas de `usuarios` (excluye `contrasena`), `UPDATE (activo)`, `EXECUTE fn_resetear_password`	Atiende cuentas sin poder leer jamás el hash bcrypt, lo que evita ataques de descifrado fuera de línea.
`rol_reportes`	`SELECT` solo sobre vistas y la vista materializada	Análisis gerencial sin acceso a tablas transaccionales ni a datos sensibles.
`rol_auditoria`	Membresía en `pg_monitor`	Monitoreo del motor sin acceso a datos ni a `pg_authid.rolpassword`.
`app_web` (LOGIN)	Hereda lectura y escritura; límite de 20 conexiones	Cuenta de servicio sin DDL ni administración.
`admin_datos` (LOGIN)	`ALL` sobre los objetos de `public`; `CREATE` en el esquema; límite de 5 conexiones	Administra la base sin el atributo `SUPERUSER`.

Las funciones sensibles son `SECURITY DEFINER` con `search_path = pg_catalog, pg_temp` y `EXECUTE` revocado a `PUBLIC`; así, la verificación de credenciales ocurre dentro del motor sin exponer la columna de contraseñas a ningún rol de aplicación.

4.3 Solución documental (MongoDB) frente a relacional (PostgreSQL)

Criterio	PostgreSQL	MongoDB
Modelo	Pedido normalizado en `pedidos`, `detalles_pedido`, `clientes` y `productos`, con integridad referencial.	Pedido autocontenido con cliente y líneas embebidos; el precio y los datos del cliente quedan congelados al momento de la compra.
Lectura del pedido completo	Combinación de 3–4 tablas (*Nested Loop*/*Hash Join*) y `DISTINCT` en la búsqueda por producto. Con índice por PK y páginas en caché, el informe de U7 cita una resolución en menos de 1 ms.	Un único `find` por `id_pedido`, sin `$lookup`; el índice multiclave resuelve la búsqueda por producto.
Consistencia	ACID nativo sobre todo el catálogo.	Atomicidad garantizada por documento; transacciones multidocumento desde la versión 4.0.
Reglas de negocio	Declarativas: `CHECK`, FK y columnas generadas.	`$jsonSchema` y `$expr`, que se evalúan en cada escritura.

Decisión: PostgreSQL se mantiene como sistema de registro para el catálogo, los clientes, el inventario y la facturación, áreas que exigen FK estrictas, roles granulares y consistencia inmediata ante escrituras concurrentes. MongoDB se adopta como almacén de lectura para pedidos cerrados, cuyo patrón dominante es leer el documento completo. Con 5 pedidos no se realizó una medición comparativa de rendimiento, por lo que la ventaja de lectura se fundamenta en el modelo de acceso y no en un *benchmark*. SQLite completa la arquitectura como base desconectada sin autoridad: no descuenta stock ni confirma ventas.

5. Declaración de Uso de IA (DUIA) y Bitácora

5.1 Herramientas utilizadas

Herramienta	Unidades	Uso
OpenCode con el modelo **Gemini**	U4 a U8	Asistente principal: análisis de clausuras y FNBC, borradores de *triggers* PL/pgSQL y de scripts de medición (U4); preparación del laboratorio y del simulacro de seguridad, con interacciones registradas en `BITACORA.md` (U5); generación de `backup_foodstore.ps1` y `restore_test.ps1` a partir de `SPEC_U6.md` (U6); consulta arquitectónica PostgreSQL frente a MongoDB (U7); esquema `STRICT`, semáforo y revisión estática de los scripts de la cursada (U8).
Kiro	U5, U8	Redacción de las especificaciones formales (*specs*): `docs/kiro_roles_spec` (RF-01 a RF-07, diseño y verificación de permisos) y `docs/kiro/` (REQ-01 a REQ-04 en notación EARS, `design.md` y `tasks.md`).
Antigravity	Este informe	Estructuración y redacción a partir de las evidencias del repositorio; cada cifra se contrastó con su archivo fuente.

5.2 Prompts y especificaciones principales

- **U5 (OpenCode, Parte C.4):** análisis del incidente usando exclusivamente el log anonimizado, en una sesión limpia y sin explicar el escenario previo.
- **U6 (OpenCode):** `SPEC_U6.md` como especificación escrita de la que partieron los scripts, revisados antes de su ejecución.
- **U7 (OpenCode):** «Actuá como Arquitecto de Bases de Datos senior [...] Evaluá técnica y comparativamente si conviene mantener PostgreSQL o migrar este subconjunto a una colección documental en MongoDB usando un modelo embebido. Contrastá rendimiento de lectura, consistencia transaccional y complejidad operativa.»
- **U8 (Kiro):** especificación con REQ-01 a REQ-04 en notación EARS. **(OpenCode):** pedido de auditoría estática de solo lectura sobre la totalidad de los scripts de la cursada.

5.3 Análisis de propuestas aceptadas y descartadas

Unidad	Propuesta de la IA	Decisión	Justificación técnica
U4	Descomposición FNBC sobre C → D y tres *triggers* de sincronización	Aceptada	Verificada con `EXCEPT` (reunión sin pérdida y equivalencia del Top 5).
U5	`GRANT ALL ON ALL TABLES` a la cuenta de aplicación «para evitar errores de permisos»	Descartada	Viola el mínimo privilegio, habilita `DELETE` / `TRUNCATE` contra la regla R7 y expone los hashes de `usuarios`.
U5	`GRANT SELECT ON usuarios TO rol_soporte`	Descartada	Expone `contrasena`; se reemplazó por privilegios por columna.
U5	Bloqueo automático de la IP atacante	Descartada	Riesgo de denegación de servicio a usuarios legítimos detrás de NAT o proxy.
U5	Comparar visualmente las tablas original y anonimizada	Descartada	Se exigió verificación exacta con `EXCEPT ALL` bidireccional.
U6	`VACUUM FULL` / `REINDEX` periódicos	Descartada	`AccessExclusiveLock` bloquea lecturas y escrituras; el diagnóstico mostró `n_dead_tup = 0`. Se adoptó un `ANALYZE` focalizado.

U6	Comparación de metadatos JSON como texto; `dropdb` automático; contraseñas en scripts	Descartadas	Falsos positivos por orden de claves; riesgo de destrucción accidental; exposición de credenciales (se usó `-w` + `pgpass.conf`).
U6	Formato `-Fc`, volcados fuera del repositorio, suite simulada de pruebas	Aceptadas	Restauración selectiva, protección de datos sensibles y validación sin motor real.
U7	«MongoDB no ofrece garantías ACID» y la migración «reduce a la mitad la latencia»	Refutada	Afirmación desactualizada: atomicidad por documento y transacciones multidocumento desde 4.0; PostgreSQL resuelve por PK en caché en menos de 1 ms.
U7	MongoDB no puede forzar total = Σ subtotales	Refutada	Demostrado con `$expr` (rechazo con código 121) en `prueba_validacion_expr.js`.
U8	La tablet descuenta stock y emite solicitudes `APROBADO`	Descartada	Viola «el móvil propone, el servidor dispone»: dos tablets desconectadas podrían vender las mismas existencias.
U8	Evaluar el conflicto de precio antes que el stock y la disponibilidad	Descartada	Un producto sin stock o dado de baja debe rechazarse; el rojo prevalece sobre el amarillo.
U8	Esquema `STRICT` con precios en centavos enteros; hallazgo F07 (recarga con `DELETE` rompe FK)	Aceptadas	Evita errores de punto flotante; la recarga se rediseñó con `UPSERT`.
U8	Hallazgo F09: un producto dado de baja seguiría disponible en la tablet	Descartada	`v_snapshot_movil` exporta los inactivos con `disponible = 'f'`; tras el `UPSERT`, la tablet los oculta.

En todas las unidades, el criterio de aceptación fue la verificación empírica sobre copias de laboratorio. Ninguna sugerencia se incorporó sin una prueba reproducible que la respaldara.